

The Ultimate Web Development Handbook



Managing Data with State

In this lecture, we will learn about state in React. While props let us pass data into components, state allows components to manage their own data and update the user interface when that data changes.

What is State?

State is like a variable that belongs to a component. Unlike normal variables, when state changes, React automatically re-renders the component to reflect the new data.

The useState Hook

React provides a special function called `useState` to create and manage state. It is called a “hook” because it hooks into React features.

Example:

```
import { useState } from "react"

function App() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click Me</button>
    </div>
  )
}

export default App
```

How it works:

- `useState(0)` creates a state variable named `count` with an initial value of `0`.
- `setCount` is the function used to update the state.
- Whenever `setCount` is called, React re-renders the component with the new state value.

Multiple State Variables

You can have more than one state variable inside a component:

```
function Profile() {  
  const [name, setName] = useState("Harry")  
  const [age, setAge] = useState(19)  
  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>Age: {age}</p>  
    </div>  
  )  
}
```

Each call to `useState` creates a separate piece of state.

Recap

- State is used to store data inside a component.
- The `useState` hook provides a way to declare and update state.
- Updating state automatically re-renders the component.

Handling Events in React

In this lecture, we will learn how to handle user interactions in React using events. Events make applications interactive, such as responding to clicks, typing, or form submissions.

Event Handling in React

React events are very similar to DOM events in plain JavaScript, but there are a few key differences:

- Event names are written in **camelCase** (e.g., `onClick` instead of `onclick`).
- You pass a function as the event handler, not a string.

Example: Button Click

```
function App() {  
  function handleClick() {  
    alert("Button was clicked!")  
  }  
  
  return (  
    <div>  
      <button onClick={handleClick}>Click Me</button>  
    </div>  
  )  
}  
  
export default App
```

Here, when the button is clicked, the `handleClick` function runs.

Inline Event Handlers

You can also define the function inline:

```
<button onClick={() => alert("Inline click!")}>Click Inline</button>
```

While this works, using separate functions is better for readability when the logic gets more complex.

Example: Updating State with Events

Let's combine events with state:

```
import { useState } from "react"

function App() {
  const [text, setText] = useState("Hello")

  function updateText() {
    setText("You clicked the button!")
  }

  return (
    <div>
      <p>{text}</p>
      <button onClick={updateText}>Change Text</button>
    </div>
  )
}

export default App
```

Clicking the button updates the state, and React re-renders the UI.

Recap

- React event handlers use camelCase names like `onClick` and `onChange` .
- Event handlers are functions that run in response to user actions.
- Combining events with state allows us to create dynamic, interactive components.

Rendering Lists and Using Keys

In this lecture, we will learn how to display lists of data in React. Often, you will need to render multiple items, such as a list of products, users, or tasks. React makes this easy using the JavaScript `map` function.

Rendering a List

Suppose we have an array of fruits. We can render them like this:

```
function App() {  
  const fruits = ["Apple", "Banana", "Orange"]  
  
  return (  
    <ul>  
      {fruits.map(fruit => (  
        <li>{fruit}</li>  
      ))}  
    </ul>  
  )  
}  
  
export default App
```

Here, we loop through the `fruits` array using `map` and return a `<li>` for each fruit.

The Key Prop

When rendering lists, React requires a special `key` prop. Keys help React identify which items have changed, been added, or removed. Without keys, React may re-render inefficiently.

```
function App() {
  const fruits = ["Apple", "Banana", "Orange"]

  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  )
}
```

In this example, we use the array index as the key. For real applications, it's better to use a unique ID if available.

Example: Rendering Objects

```
function App() {
  const users = [
    { id: 1, name: "Alice" },
    { id: 2, name: "Bob" },
    { id: 3, name: "Charlie" }
  ]

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  )
}
```

Each user has a unique `id`, which is perfect for the `key` prop.

Recap

- Use the JavaScript `map` function to render lists in React.
- Always add a unique `key` prop to list items.
- Keys help React efficiently update the UI when lists change.

Conditional Rendering in React

In this lecture, we will learn how to show or hide parts of the user interface based on conditions. This is called conditional rendering.

Using if Statements

You can use a normal JavaScript `if` statement inside a component:

```
function App() {  
  const isLoggedIn = true  
  
  if (isLoggedIn) {  
    return <h2>Welcome back!</h2>  
  } else {  
    return <h2>Please log in.</h2>  
  }  
}  
  
export default App
```

Here, the UI changes depending on the value of `isLoggedIn`.

Using the Ternary Operator

A shorter way to write conditional logic is the ternary operator:

```
function App() {  
  const isLoggedIn = false  
  
  return (  
    <div>  
      {isLoggedIn ? <h2>Welcome back!</h2> : <h2>Please log in.</h2>}  
    )  
  )  
}
```

```
    </div>
  )
}
```

Using Logical AND (&&)

When you only want to render something if a condition is true, you can use the logical AND operator:

```
function App() {
  const showMessage = true

  return (
    <div>
      <h1>Hello User</h1>
      {showMessage && <p>This is a special message!</p>}
    </div>
  )
}
```

If `showMessage` is `false`, the paragraph will not be rendered.

Example: Toggle Visibility

Let's combine state and conditional rendering:

```
import { useState } from "react"

function App() {
  const [visible, setVisible] = useState(true)

  return (
    <div>
      <button onClick={() => setVisible(!visible)}>
        {visible ? "Hide" : "Show"}
      </button>
    </div>
  )
}
```

```
    {visible && <p>This text can be toggled on and off.</p>}  
  </div>  
)  
}  
  
export default App
```

Clicking the button toggles the paragraph on and off.

Recap

- Use `if` statements, the ternary operator, or logical AND (`&&`) for conditional rendering.
- Conditional rendering allows the UI to change based on state or props.
- You can combine state and conditions to build interactive features.

Forms and Two-Way Binding

In this lecture, we will learn how to handle forms in React. Forms allow users to input data, and with two-way binding, we can keep the input value in sync with component state.

Controlled Components

In React, form inputs are usually controlled by state. This means the value of the input is stored in state and updated on every change.

Example with a text input:

```
import { useState } from "react"

function App() {
  const [name, setName] = useState("")

  function handleChange(event) {
    setName(event.target.value)
  }

  return (
    <div>
      <input type="text" value={name} onChange={handleChange} />
      <p>Your name is: {name}</p>
    </div>
  )
}

export default App
```

Here:

- `value={name}` means the input's value comes from state.

- `onChange` updates the state whenever the user types.

This is called two-way binding: the input updates the state, and the state updates the input.

Handling Multiple Inputs

For multiple inputs, you can have separate state variables:

```
import { useState } from "react"

function LoginForm() {
  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")

  function handleSubmit(event) {
    event.preventDefault()
    alert(`Email: ${email}, Password: ${password}`)
  }

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label>Email: </label>
        <input type="email" value={email} onChange={e =>
setEmail(e.target.value)} />
      </div>
      <div>
        <label>Password: </label>
        <input type="password" value={password} onChange={e =>
setPassword(e.target.value)} />
      </div>
      <button type="submit">Login</button>
    </form>
  )
}

export default LoginForm
```

The `handleSubmit` function prevents the page from refreshing and shows the input values in an alert.

Recap

- Controlled components keep form inputs in sync with state.
- Two-way binding means user input updates state, and state updates the input field.
- Forms in React usually require event handlers for `onChange` and `onSubmit` .

Styling Components in React

In this lecture, we will learn different ways to style components in React. Styling is important to make your application look polished and user-friendly.

Using Regular CSS Files

You can write your styles in a separate CSS file and import it into your component.

App.css :

```
.container {  
  text-align: center;  
  padding: 20px;  
  border: 1px solid gray;  
}
```

App.jsx :

```
import "./App.css"  
  
function App() {  
  return <div className="container">Styled with CSS file</div>  
}  
  
export default App
```

Here, we use `className` instead of `class` because `class` is a reserved word in JavaScript.

Inline Styles

You can also apply styles directly to elements using inline style objects.

```
function App() {
  const style = {
    color: "blue",
    fontSize: "20px",
    padding: "10px"
  }

  return <h2 style={style}>Styled with Inline Styles</h2>
}

export default App
```

Notice that CSS properties are written in **camelCase** (e.g., `fontSize` instead of `font-size`).

CSS Modules

For component-specific styles, you can use CSS Modules. This prevents class name conflicts.

`Button.module.css` :

```
.button {
  background-color: green;
  color: white;
  padding: 10px;
  border: none;
  border-radius: 5px;
}
```

`Button.jsx` :

```
import styles from "./Button.module.css"

function Button() {
  return <button className={styles.button}>Click Me</button>
}
```

```
export default Button
```

Here, `styles.button` applies the scoped CSS class to the button.

Recap

- You can style React components with regular CSS files, inline styles, or CSS Modules.
- `className` is used instead of `class`.
- CSS Modules are useful to avoid class name conflicts.

Navigation with React Router

In this lecture, we will learn how to add navigation to a React app using **React Router**. This allows us to create multiple pages within a single-page application.

Installing React Router

First, install React Router with npm:

```
npm install react-router-dom
```

Setting Up Routes

Open `main.jsx` and wrap your app in a `BrowserRouter` :

```
import React from "react"
import ReactDOM from "react-dom/client"
import { BrowserRouter } from "react-router-dom"
import App from "./App"
import "./index.css"

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```

Defining Routes

Inside `App.jsx`, use the `Routes` and `Route` components to define paths.

```

import { Routes, Route, Link } from "react-router-dom"

function Home() {
  return <h2>Home Page</h2>
}

function About() {
  return <h2>About Page</h2>
}

function Contact() {
  return <h2>Contact Page</h2>
}

function App() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link> |{" "}
        <Link to="/contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  )
}

export default App

```

Here's what happens:

- The navigation bar uses `Link` to change pages without reloading.
- `Routes` and `Route` define which component shows for each path.

- For example, `/about` displays the `About` component.

Recap

- React Router enables multi-page navigation in React apps.
- Use `BrowserRouter` to enable routing.
- Define routes with `Routes` and `Route`.
- Use `Link` for navigation instead of `<a>` tags.

React Hooks: useState

In modern React, **hooks** allow you to add functionality like state and lifecycle features to functional components. The most commonly used hook is **useState**, which lets you manage state within a component.

What is useState?

- State is used to store and update dynamic values in a React component.
 - `useState` provides a way to declare and manage state in functional components.
-

Syntax

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click Me
      </button>
    </div>
  );
}
```

How it works

- `useState(0)` initializes the state variable `count` with `0`.
 - `count` is the current state value.
 - `setCount` is a function to update the value.
 - When the button is clicked, `setCount(count + 1)` updates the state, causing the component to re-render.
-

Key Points

- `useState` returns an array with **two values**: the state and the updater function.
 - Updating state with `setStateFunction` re-renders the component.
 - You can use `useState` multiple times in a component to manage different pieces of state.
-

Example with Multiple States

```
function UserInfo() {  
  const [name, setName] = useState("Alice");  
  const [age, setAge] = useState(25);  
  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>Age: {age}</p>  
      <button onClick={() => setAge(age + 1)}>Increase Age</button>  
    </div>  
  );  
}
```

When to use useState

- Whenever you need to track user input, counters, toggles, or any dynamic value inside your component.

React Hooks: useEffect

The **useEffect** hook allows you to perform side effects in React components. Side effects are tasks that run outside the normal component rendering process, such as fetching data, updating the document title, or setting up subscriptions.

What is a Side Effect?

Examples of side effects include:

- Fetching data from an API
 - Directly updating the DOM
 - Setting up timers or intervals
 - Subscribing to events
-

Syntax

```
import { useState, useEffect } from "react";

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click Me</button>
    </div>
  );
}
```

```
    </div>
  );
}
```

How it works

- `useEffect` runs after every render by default.
- In the above example, whenever `count` changes, the document title is updated.

Using Dependencies

You can control when the effect runs by passing a dependency array as the second argument.

```
useEffect(() => {
  console.log("Effect ran!");
}, [count]);
```

- The effect will only run when `count` changes.
- If you pass an empty array `[]`, the effect runs only once after the component mounts.

Cleanup with `useEffect`

Some effects require cleanup, like removing event listeners or clearing intervals. You can return a function from `useEffect` for cleanup.

```
useEffect(() => {
  const timer = setInterval(() => {
```

```
    console.log("Timer running");  
  }, 1000);  
  
  return () => clearInterval(timer);  
}, []);
```

- The cleanup function runs before the component unmounts or before the effect re-runs.
-

Key Points

- `useEffect` replaces lifecycle methods like `componentDidMount` , `componentDidUpdate` , and `componentWillUnmount` .
- Always specify dependencies clearly to avoid unnecessary re-renders.
- Use cleanup to prevent memory leaks.